

Legacy-to-Cloud-Native Assessment — Design Conversation

Project catalogue, daily multi-format reporting, air-gapped delivery

18 September 2026

Contents

About this document	1
1. Initial requirements	2
2. Known fields and what else will be needed	3
3. Hand-edited YAML and what “verify” should mean	7
4. Answering manager questions	8
5. Air gap and starting quickly	10
6. Building the importer	12
7. Handoff specification	14
8. Driving Claude Code with the spec	15
9. Naming the document	16
Summary of decisions	17

About this document

A working conversation covering the technical design of a cross-team project assessment system: a catalogue of several hundred legacy projects being migrated to cloud native, with daily status reporting in five formats, versioned data, and delivery into an air-gapped environment.

Artifacts produced during the conversation and referenced below:

File	What it is
import_xlsx.py	Working importer: legacy spreadsheet to per-project YAML (~1540 lines, tested)
README.md	Documentation for the importer
make_testdata.py	Generates a synthetic messy German legacy sheet for testing
HANDOFF.md / SPEC.md	Implementation spec for the remaining pipeline
example-profile.md	Sample column-profile output
example-import-report.md	Sample import report

1. Initial requirements

User

I need to analyse a vast number of projects, each owned and ran by a different team. The list of data elements that I gather is a moving target, it evolves during analysis. Every day I need to present the status in various formats: excel, pdf, powerpoint, png, txt. The excel is used for managers to analyse since they're used to excel. Data must be versioned. All must run in containers.

Make a first proposal for the technical approach.

Claude

The central design decision is to separate three things that people usually conflate: the record of truth, the report model, and the renderers.

Data store: Git as system of record

One file per project, YAML or JSON, in a Git repo:

```
projects/  
  team-alpha/payment-gateway.yaml  
  team-beta/legacy-crm.yaml  
schema/  
  project.schema.json          # versioned alongside
```

Why Git rather than a database: the schema is a moving target and ownership is distributed. Git gives versioning, blame, diffs, review, and CODEOWNERS per team directory for free. Teams submit merge requests against their own files; you never chase anyone for a spreadsheet. A daily tag (snapshot/2026-09-14) makes any past report reproducible.

Schema evolution rules that keep this from rotting:

- Every record carries `schema_version`.
- Additive-only changes within a major version. New fields are optional with a defined default of “unknown”, not empty.
- CI validates every MR against the JSON Schema. Invalid data never lands.
- A migration script per version bump, run as a normal commit so the change is visible in history.

Treat “field not yet collected” and “field collected, value is none” as distinct states. This bites hard in reporting if you skip it.

Derived analytical layer

On each run, load all YAML into DuckDB and write a Parquet snapshot to object storage (MinIO or S3 with object versioning). DuckDB handles the flattening of nested and sparse fields well, and gives the Excel builder plain SQL. The Parquet snapshots give a time series across days without re-walking Git history.

Store a `manifest.json` alongside each run: Git SHA, schema version, image digest, timestamp. That manifest is what makes a report auditable.

Report model: the key abstraction

Do not let five renderers each query the data independently. You will get five subtly different numbers.

DuckDB -> `report_builder.py` -> `report_model.json` -> [xlsx, pdf, pptx, png, txt]

The report model is a plain JSON document containing every number, label and table that will appear in any output, already aggregated and already formatted as strings where formatting matters. Renderers become dumb. Adding a sixth format later costs a day, not a rewrite. The model is also the artifact you diff when a manager asks why Tuesday's number changed.

Charts follow the same rule: render PNGs once with matplotlib, then embed the same files in PDF, PPTX and Excel.

Which fields appear in which report goes in a `reports/*.yaml` spec, so adding a newly collected data element to the deck is a config change. Missing fields render as “n/a” rather than crashing the run.

Renderers

- **Excel** (openpyxl or xlsxwriter): managers want to slice it themselves, so give them a flat data sheet as a real Excel table with autofilter and frozen header, plus a dashboard sheet with the charts. Resist heavy formatting on the data sheet, it fights pivot tables.
- **PDF**: Typst is a good fit for containers. Fast, single small binary, no TeX layer, proper templating. WeasyPrint is the alternative if you would rather template in HTML/CSS.
- **PPTX** (python-pptx): work from a corporate .potx and fill named placeholders. Do not build slides from scratch, and do not use LibreOffice headless for conversion unless you have no choice.
- **PNG**: matplotlib, one chart per file, deterministic filenames.
- **TXT**: Jinja2. Cheap and surprisingly valuable, because it diffs cleanly day over day.

Containers

Two images:

- collector: Python plus whatever API clients you need for Jira, GitLab, etc.
- reporter: Python, DuckDB, Typst, matplotlib, and the fonts. Fonts and `de_DE.UTF-8` locale are the actual pain here, so pin them and pin the image by digest.

Pin renderer versions tightly. Typst and matplotlib both shift layout between minor versions, which breaks byte-level reproducibility of the PDFs.

Orchestration

Start with a scheduled GitLab CI pipeline: validate, build snapshot, build report model, render, publish to MinIO under `reports/YYYY-MM-DD/`. It is the smallest thing that works and the runner already exists. Move to Argo Workflows only when fan-out across many collectors or per-team parallelism is needed. Skip Airflow.

Open questions

Three things would change the shape of this: whether teams will actually edit YAML in merge requests or need a form/UI in front of it; whether the daily reports go to a fixed audience or need per-team scoping; and whether “versioned” means “reproduce any past report” or also “query history as a time series”.

2. Known fields and what else will be needed

User

Editing txt/yaml files should be possible. A UI would be of course good, too.

List of known required data fields:

- location of dc
- list of team members
- classification-A, values CA-1, CA-2, CA-3
- classification-B, values CB-1, CB-2, CB-3, CB-4
- database used: MSSQL, DB2, AIX, redis
- security class: S1, S2, S3, S4
- storage classes required: block, object
- storage size needed

- CPUs needed

Imagine more software related fields. Also suggest more fields/data types etc. you think will be necessary in future. Task is to go from legacy to cloud native.

Claude

Two things before the field list.

AIX isn't a database. It's IBM's Unix, and it usually shows up next to DB2 on Power hardware. That distinction matters a lot here: a DB2-on-AIX/ppc64le workload is a completely different migration problem than DB2 on x86. So OS, CPU architecture, and datastore engine need to be three separate fields. If the intake list has them merged, other fields are probably merged too, and that's worth a pass before freezing v1.

Every field should drive a decision. For legacy-to-cloud-native, each field should answer one of: what does it cost (sizing), what stops us (blockers), when do we move it (sequencing), or who does it (ownership). Fields that don't map to one of those are noise that teams will resent filling in, and coverage will stall around 40%.

Worked record

Enum codes are references into a separate taxonomy file, not baked into the schema.

```
schema_version: 1
id: payment-gateway           # immutable slug, never reuse
name: Payment Gateway
lifecycle: active             # planned|active|maintenance|sunset|decommissioned
business_tier: tier-1         # drives migration order, not the same as security
↪ class

ownership:
  team_id: team-alpha         # -> teams.yaml
  contacts:
    - role: tech_lead         # tech_lead|product_owner|ops|security|architect
      person_id: m.huber
    - role: ops
      person_id: s.bauer
  cost_center: "4711"
  vendor: in-house           # in-house|cots|cots-customised <- decides if
↪ refactor is even legal

classification:
  classification_a: CA-2      # -> taxonomy.yaml
  classification_b: CB-3
  security_class: S3
  compliance_scope: [gdpr, bafin]
  data_residency: de-only    # de-only|eu-only|any <- can kill a target
↪ platform outright

placement:
  environments:
    - name: prod
      datacenter: muc-01      # -> sites.yaml (site code, not free-text city)
      deployment_model: vm    # bare-metal|vm|container|mainframe
      instances: 4
    - name: staging
      datacenter: muc-01
      deployment_model: vm
      instances: 1

platform:
```

```

os: { family: rhel, version: "7.9", eol: 2024-06-30 }
cpu: { cores: 16, architecture: x86_64, peak_utilisation_pct: 35 }
memory_gb: 64
storage:
  - class: block # block|object|file
    size_gb: 500
    iops: 3000
    growth_pct_per_year: 15
  - class: object
    size_gb: 2000
middleware:
  - { product: jboss-eap, version: "7.2", eol: 2025-11-30 }

datastores:
  - engine: mssql #
    ↪ mssql|db2|oracle|postgres|mysql|redis|mongodb|other
    version: "2016"
    size_gb: 800
    ha_model: always-on
    licence_model: per-core # per-core|per-socket|subscription|open-source
    managed_target: azure-sql # what it should become, "unknown" until decided

software:
  languages: [{ name: java, version: "8" }]
  repo_url: https://gitlab.../payment-gateway
  in_version_control: yes # yes|no|partial <- surprisingly often "no"
  build_system: maven
  ci_pipeline: yes
  deploy_frequency: monthly # per_commit|daily|weekly|monthly|quarterly|manual
  automated_tests: partial
  test_coverage_pct: 41

cloud_native_readiness: # all tri-state: yes|no|partial|unknown
  config_externalised: partial
  stateless: no
  logs_to_stdout: no
  health_endpoints: no
  graceful_shutdown: unknown
  horizontal_scaling: no
  local_filesystem_writes: yes
  sticky_sessions: yes
  host_cron_jobs: yes
  hardcoded_hostnames: yes
  secrets_in_code: partial
  auth_mechanism: kerberos # kerberos|ldap|ad|saml|oidc|basic|none
  containerised: none # none|dockerfile|image_built|runs_in_k8s
  iac_status: none # none|partial|full
  observability: { logs: file, metrics: none, tracing: none }

integration:
  depends_on: [customer-master, fraud-check] # builds the dependency graph
  consumed_by: [webshop]
  styles: [soap, mq, file-drop] # file-drop and db-link are migration
  ↪ pain
  external_partners: [sparkasse-clearing]

operations:
  availability_target: "99.9"
  rto_hours: 4
  rpo_minutes: 15

```

```

support_model: 24x7
maintenance_window: "Sun 02:00-06:00 CET"
peak_rps: 120
batch_windows: ["nightly 23:00-01:00"]

migration:
  strategy: replatform #
  ↪ rehost|replatform|refactor|repurchase|retire|retain|relocate
  target_platform: kubernetes # kubernetes|paas|saas|vm|retain
  target_site: unknown
  wave: unassigned
  target_quarter: unknown
  status: assessed #
  ↪ not_assessed|assessed|planned|in_progress|migrated|verified|rolled_back
  effort: L # S|M|L|XL
  blockers:
    - id: BLK-014
      type: licensing #
    ↪ licensing|technical|compliance|vendor|people|budget
      description: MSSQL licence is bound to physical cores
      severity: high
      owner: m.huber
      status: open

_meta:
  last_reviewed: 2026-09-10
  reviewed_by: m.huber
  confidence: estimated # verified|estimated|unknown
  notes: |
    CPU figures from the 2024 capacity review, not re-measured.

```

Type decisions worth arguing about now

No booleans. Use yes|no|partial|unknown. A boolean forces a team to guess, and false is indistinguishable from “nobody asked yet”. Completeness metrics depend on being able to tell those apart.

unknown is a legal value everywhere, and coverage is a headline report. “Team Beta: 38% of fields assessed” on slide two does more to get data filled in than any reminder email.

Enums live in taxonomy.yaml, not in the JSON Schema. Each code gets a label (German and English), a description, a sort order, and a colour for charts. Adding CB-5 is then a one-line data change, and the PPTX legend updates itself. The CA/CB/S codes are opaque to everyone outside the room where they were invented, so the report layer must be able to print the label.

Units in field names, always: size_gb, rto_hours, memory_gb. Never a bare storage: 500.

Confidence at block level, not field level. Per-field provenance is correct and nobody maintains it. _meta.confidence plus last_reviewed is the compromise that survives contact with reality.

Never store computed scores. Readiness score, risk score, blocker count all belong in the report builder. In the YAML they go stale the moment someone edits a field.

depends_on is the field that decides the project plan. Migration waves fall out of the dependency graph, not out of a spreadsheet sort. Make it mandatory, allow [], and render the graph as a PNG from day one. It also finds the cycles, which is where the real work is.

Fields you’ll wish you had added later

Licence bindings (per-core, dongle, MAC-address-pinned), because they turn a rehost into a repurchase. Vendor support posture for containers, because a COTS product unsupported on Kubernetes ends the discussion. Hardcoded IPs and hostnames. Shared filesystem mounts

between projects, which is the dependency nobody documents. Certificate inventory and expiry. Whether the build can even be reproduced today, because a surprising number of legacy binaries have no working build.

The UI

Keep Git as the store and make the UI a client of it. The critical rule: **generate the form from the JSON Schema**, don't hand-write it. With a moving-target schema, any hand-built form is a second place to update and it will drift within three weeks.

Minimal version: FastAPI plus a JSON-Schema-driven form renderer (JSON Forms or react-jsonschema-form), authenticated against the existing IdP, writing through the GitLab API to a branch and opening a merge request. Both paths, editor and UI, then land in the same validation pipeline and the same history.

If you'd rather buy than build, Backstage is the obvious candidate: catalog YAML in Git, ownership model, dependency graph, and a scaffolder for the forms. It's heavier than the problem needs right now, but it's where this ends up if the catalog outlives the migration.

3. Hand-edited YAML and what “verify” should mean

User

My idea was to hand-edit yaml files and have a script that verifies syntax. What's your proposal? UI only?

Claude

Your idea is the right one. Don't build a UI now.

The reason isn't that a UI is bad, it's that the store and the editor are separable decisions. Git stays the system of record either way, and any UI is just a client that opens merge requests. So you lose nothing by deferring it, and you avoid building a form for a schema that's still moving. Build the UI when a specific group of people is provably blocked, not before.

But “a script that verifies syntax” is aiming too low. Syntax errors are the cheapest problem you have.

Five layers, one script

1. Parse. Use `ruamel.yaml` rather than `PyYAML`, for two reasons: it gives line and column numbers for every node, and it catches duplicate keys. `PyYAML` silently keeps the last one, which means a team can have two storage: blocks and the data quietly wrong forever.

2. Structure. JSON Schema validation. Unknown fields rejected, so a typo like `secru-tiy_class` fails instead of vanishing.

3. Referential integrity. This is where the real bugs live and where a syntax checker gives you nothing. `team_id` resolves against `teams.yaml`. `datacenter` resolves against `sites.yaml`. Every entry in `depends_on` names a project that exists. The dependency graph has no cycles. Enum codes exist in `taxonomy.yaml`.

4. Plausibility. Soft rules: a project with `deployment_model: container` but `containerised: none`, a tier-1 project with `rto_hours: 168`, block storage declared with size 0, no prod environment. These warn rather than fail.

5. Completeness. Not a pass/fail check, a number. Percentage of assessed fields per project and per team, written to the daily report.

Hard rule: **incomplete never fails, invalid always fails.** If a team can't commit a half-filled file, they'll keep the data in their own spreadsheet and you will never see it.

A YAML trap

Earlier I suggested `yes|no|partial|unknown` as values. In YAML 1.1, which PyYAML and most tooling still implement, bare `yes` and `no` parse as booleans. So `in_version_control: no` becomes `False`, and `stateless: no` doesn't compare equal to the string the report expects. The Norway problem, same family.

Drop the `yes/no` vocabulary entirely rather than quoting around it:

```
config_externalised: partial      # full | partial | none | unknown
stateless: none
in_version_control: full
```

Same class of trap: version numbers. `version: 7.9` is a float that loses trailing zeros, `version: "2016"` unquoted is an integer, `os_version: 1.2.3` is a string but `1.2` isn't. Force type: `string` and pattern in the schema for every version field so the validator catches it instead of a PDF showing `7.900000000000000004`.

What replaces the UI

Editor integration. First line of every project file:

```
# yaml-language-server: $schema=../../schema/project.schema.json
```

VS Code with the Red Hat YAML extension, or neovim with `yamlls`, then gives autocompletion of enum values, inline descriptions on hover, and errors as you type. A team member typing `security_class:` gets `S1` through `S4` offered with their meanings. That's most of what a form would have given you, for the cost of one line and a well-written schema with description on every field.

A generator. `make new-project TEAM=alpha NAME=payment-gateway` writes a skeleton with every field present, every value unknown, and a comment above each one explaining it. Nobody starts from a blank file or copies a stale example.

Error messages that name file, line, and fix. The single highest-leverage piece, and usually an afterthought. Not `ValidationError: 'CB-7' is not one of [...]` but:

```
projects/team-alpha/payment-gateway.yaml:14:22
  classification_b: CB-7 is not a known code
  valid: CB-1, CB-2, CB-3, CB-4 (see taxonomy.yaml)
```

Same script runs in three places with one code path: `make check` locally, a pre-commit hook, and CI on every merge request.

When to revisit the UI

Add one when you can name the population that's blocked. Usually it's ops staff without Git access, external vendors, or the people who own the licence and contract data and will never touch a merge request.

One thing worth doing earlier than an editing UI is a read-only view, since "what does the catalog currently say about my project" is asked ten times more often than anyone edits.

4. Answering manager questions

User

Managers will want to know stuff like: how many projects have DB2; how many projects have status 'retain'; how many projects in DCx run windows. How shall we accomplish that? Via excel?

Claude

Excel will answer all three, and it's where the managers already are. But those three questions are different from each other in a way worth separating.

The three question types

Recurring and known. "How many projects have DB2" gets asked every week by the same people. These belong in the daily deck as a pre-built chart or table. If a manager has to open a file and build a pivot to get a number you already know they want, the reporting has failed.

Recurring but sliced differently each time. "How many projects in DCx run Windows" is really "show me the cross-tab of site against OS." Nobody wants twelve variants of that slide. This is what a pivot table is for, and why Excel is the right answer for this class.

One-off. "How many tier-1 projects have DB2 on AIX in Munich with an open licensing blocker" gets asked once in a steering meeting and never again. You'll answer it with a DuckDB query in thirty seconds. Don't try to anticipate these.

Making the Excel actually pivotable

The failure mode is shipping a pretty formatted sheet that pivot tables choke on.

One row per project, flat, no merged cells, no blank spacer rows, header in row 1. Declared as a real Excel table (`ws.add_table`) so pivots and formulas reference it by name and expand when rows are added.

The problem is that the data isn't flat. A project has several environments, several datastores, several storage classes. Flatten it wrong and every count is inflated. Use multiple sheets at different grains, each a proper table:

- projects — one row per project. All single-valued fields. Answers "how many projects have status retain" directly.
- datastores — one row per project-datastore pair. Answers "how many projects have DB2" via a distinct count of project ID, not a row count.
- environments — one row per project-environment. Site, OS, deployment model, instance count. Answers the DCx-Windows question, and note it's a per-environment question, since a project can have prod in Munich and staging in Frankfurt.
- blockers — one row per blocker.
- dependencies — one row per edge.

On the projects sheet, also carry flattened convenience columns computed by the builder: `datastore_engines: "db2, redis", has_db2: yes, sites: "muc-01", os_families: "windows"`. Managers filter on those without understanding the grain question. The multi-grain sheets are there for when they need to be correct.

Two columns per coded field. `security_class_code = S3` and `security_class_label = "Vertraulich"`. Pivot on the code so sort order is right, display the label.

Ship the pivots pre-built. A pivots sheet with three or four ready cross-tabs (site x OS, strategy x business tier, engine x migration status). Most managers won't create a pivot from scratch but will happily change the row field on one that exists. `openpyxl` can't create pivot tables, so the practical route is a `template.xlsx` containing the pivots and empty tables, into which you write data each day with the pivot cache set to refresh on open.

Don't make Excel the only path

The daily deck carries the top ten numbers. Engine counts, status distribution, site x OS matrix, blocker count by type, readiness by team. If three managers ask the same question twice, it earns a slide.

A counts sheet, plain and boring. Every single-field frequency table stacked vertically. No pivot required, no Excel skill required. This sheet answers probably 70% of what gets asked, takes fifteen lines of DuckDB `GROUP BY`, and is what you paste into an email.

One warning about counts

“How many projects have DB2” has at least three defensible answers: projects with any DB2 instance, projects where DB2 is the primary datastore, and total DB2 instances. Managers will assume whichever suits their point and then two slides will disagree.

Define the counting rules once in the report builder, apply them everywhere, and put the definition in the file. A small definitions sheet, one line each: “Projects with DB2: distinct projects having at least one datastore with engine=db2, all environments included.” Unglamorous, and it ends the meetings where two people argue about whose number is right.

Also decide now how unknown is counted. If 40% of projects haven’t declared an OS, “how many run Windows” has a denominator problem, and the honest chart shows the unknown bar rather than hiding it.

5. Air gap and starting quickly

User

The platform where this code is used will be air gapped, but I want to start quickly without days of manual copying data from an existing excel. What could we do?

Claude

Two separate problems: getting the legacy Excel in, and making the whole thing run without a network. The first is where the days go.

Assume the Excel can’t leave

Design so no data crosses the boundary. Build the importer outside against an anonymised sample (same columns, scrambled values, twenty rows), ship the importer inside the bundle, run it on the real file there. The classification question never blocks you and you don’t have to ask permission for anything.

That constrains the importer: dependency-light, and readable diagnostics, because you won’t be debugging it interactively with the real data in front of you.

The importer, in two stages

Stage 1: profile, don’t map. First run emits a report, not YAML. For every column: fill rate, distinct value count, and the full list of distinct values when there are fewer than fifty. You’ll get something like Datenbank: 14 distinct values with MSSQL, MS-SQL, SQL Server 2016, mssql/db2, keine, ?, empty.

That list is what you review, once. Reviewing 200 distinct values takes an hour. Reviewing 800 rows takes days. This is the whole trick.

Stage 2: mapping files you edit by hand.

```
# mapping.yaml
"Standort RZ":      placement.environments[0].datacenter
"Datenbank":       datastores[].engine
"CPU":             platform.cpu.cores
"Sicherheitsklasse": classification.security_class

# value_map.yaml
datastores.engine:
  "MS-SQL": mssql
  "MSSQL": mssql
  "SQL Server 2016": mssql
  "mssql/db2": [mssql, db2] # split cells are normal in legacy sheets
```

```
"keine": null
"?": unknown
```

Both are data, both live in Git, both are reviewable. When the sheet gets updated you adjust a line rather than the code.

Nothing gets dropped. Any column you haven't mapped goes into the project file verbatim:

```
_unmapped:
  "Bemerkung": "Migration 2023 gestoppt, Lizenzthema"
  "Ansprechpartner alt": "H. Weber (ausgeschieden)"
```

This is what lets you start in two days instead of two weeks. Map the fifteen columns you understand, the other thirty ride along untouched, and you promote them into the real schema as you work out what they mean. The validator must allow `_unmapped` with arbitrary content, and the completeness report should list the most common unmapped keys so you can see what's worth promoting.

Mark the provenance.

```
_meta:
  source: legacy-xlsx
  imported_at: 2026-09-14
  confidence: imported           # verified | estimated | imported | unknown
  last_reviewed: null
```

An import gives you 90% field coverage that means almost nothing. If you don't separate "imported, nobody has looked at it" from "a team member confirmed it", your first status report will be confidently wrong. Report both numbers: coverage and verified coverage. The gap between them is the actual work remaining, and it's a good slide.

Make re-import a Git merge. The Excel will keep being edited for a few weeks after you start. Run the importer onto a dedicated import branch each time and merge into main. Fields humans have edited show up as conflicts, everything else fast-forwards. Stable IDs matter for this, so generate slugs once and keep `id_map.csv` alongside, keyed by whatever the sheet's stable identifier is.

Air gap: the failure modes are boring and specific

Build a single bundle outside, verify it, carry it in.

Contents: container images as OCI archives (skopeo copy docker://... oci-archive:..., pinned by digest), the repo as a git bundle so history comes along and can be pushed to the internal GitLab, the schema and taxonomy, fonts, and a manifest with checksums.

Things that reach for the network when you don't expect it:

- **DuckDB downloads extensions on first use.** parquet is built in, but httpfs, spatial or excel get fetched. Pre-install into the image and set `extension_directory` and `autoinstall_known_extensions=false`.
- **Typst pulls packages from its registry** for any `#import "@preview/..."`. Vendor them into the local package directory or write templates without external packages.
- **matplotlib** builds a font cache on first run. Bake the cache into the image, set `MPLCONFIGDIR`, and install the fonts you actually use.
- **Fonts generally.** Corporate PPTX templates reference fonts that may not be in the image. Missing fonts don't error, they silently substitute, and the slides look wrong in a way nobody reports for a month.
- **Python wheels.** Install everything at image build time.
- **CA certificates** for the internal GitLab, and `de_DE.UTF-8` plus timezone.

The discipline that catches all of these: a CI job outside that runs the complete pipeline with `--network=none` against the synthetic sample. If it passes there, it passes inside. Without that job you'll discover each item above one at a time, on the wrong side of the boundary, with a slow feedback loop.

Sequence

Profile the sheet and write the mappings first, since everything else is downstream of knowing what's in there. Then importer plus `_unmapped` passthrough, and generate the schema from what you found rather than from guesses. Validator next, and only then the report builder. First Excel out of the pipeline within a week, populated with imported data, honestly labelled as unverified.

The one thing not to defer: decide today whether the Excel stays authoritative during the transition. If both the sheet and the repo are live for more than a couple of weeks, they will diverge and you'll spend longer reconciling them than the original import would have cost. Set a cutover date, announce it, and after it the sheet is read-only history.

6. Building the importer

User

I can't get more specific about fields. The excel has one line per project, fields are columns. Can you write a program that creates yaml files from that excel. Target is to minimise manual work but we can't nullify that of course.

Claude

(Claude wrote and tested `import_xlsx.py`, `README.md` and `make_testdata.py`. The design and the bugs found during testing are summarised below; the full source is delivered as separate files.)

What the program does

Three subcommands:

```
python import_xlsx.py profile      projekte.xlsx --out import/
# review import/profile.md, edit mapping.yaml and value_map.yaml
python import_xlsx.py convert      projekte.xlsx --config import/ --out projects/
python import_xlsx.py derive-schema projekte.xlsx --config import/ --out
    ↪ schema/project.schema.json
```

Dependencies: `openpyxl` and `PyYAML`. No network access anywhere in the code.

Auto-detected: header row (including sheets that start with a title block), two-row merged headers, column types (integer, number, date, enum, list, free text), units in headers (Storage (GB) becomes `storage_gb`), multi-value separators, the id column, the grouping column, and CSV encoding (utf-8, cp1252, latin-1) and delimiter.

Mapping syntax

Path syntax	Result
<code>a.b.c</code>	nested scalar
<code>a.b[]</code>	list of scalars
<code>a.b[].c</code>	list of objects, one per token in a multi-value cell
<code>a.b[0].c</code>	explicit index
<code>null</code>	leave the column in <code>_unmapped</code>

Several columns can feed one list. `datastores[].engine` and `datastores[].version` are zipped by token position, so a row with "DB2; Redis" and "11.5; 7.0" produces two complete entries. Mismatched token counts are padded and reported.

Three things that reduce the manual work

Spelling variants collapse automatically. MS-SQL, MSSQL and mssql are clustered by an aggressive normalisation key and all proposed as the most frequent spelling. Near-misses the tool won't merge on its own appear as comments for a human to decide.

Not-set markers are proposed as null (keine, unbekannt, offen, k.A.).

High-cardinality columns are skipped in value_map.yaml. Generating 800 entries for a project-number column is noise; they're listed at the bottom as comments instead.

Sample output

```
schema_version: 1
id: p-1002
legacy_id: P-1002
name: Legacy CRM
classification:
  classification_a: CA-3
  classification_b: CB-4
  security_class: S1
datastores:
- engine: mssql
migration:
  status: not_assessed
  strategy: retire
platform:
  os: SLES 15
  os_eol: '2024-06-30'
  storage_gb: 999
  storage_classes:
  - block
_unmapped:
  Ansprechpartner: S. Bauer
  Bemerkung: Migration 2023 gestoppt, Lizenzthema
_meta:
  source: projekte.xlsx
  source_sheet: Projektuebersicht
  source_row: 5
  imported_at: '2026-09-14'
  confidence: imported
  last_reviewed: '2026-09-12'
  reviewed_by: m.huber
```

Two bugs found during testing, both silent data corruption

2.500 is 2500 in a German sheet and 2.5 in an English one, and a single cell carries no evidence either way. The first version silently read it as 2.5. The fix decides per column from the other values in it; when a column gives no evidence, it says so loudly in profile.md and defaults to German. --decimal de|en overrides.

A decimal comma in 1.200,5 was being detected as a multi-value separator and splitting the cell into three. Number and date detection now runs before list detection.

Re-import behaviour

Tested against a simulated workflow: a human marks one project as verified, one cell in the sheet changes, re-import runs.

- Exactly one file is rewritten. The other nineteen are untouched, so git status shows real changes rather than 800 timestamps.
- Ids are stable via import/id_map.csv.

- `_meta.last_reviewed` and `reviewed_by` are carried over.
- When the underlying data changed, `_meta.confidence` drops back to `imported`, because verifying the old values no longer means anything.
- Volatile provenance (file checksum, import date) lives in `projects/_import_manifest.yaml`, not in every project file.

What to watch after the first run

The gap between coverage and verified coverage. The import will hand you something like 90% field coverage that nobody has looked at, and that number will get quoted in a steering meeting if you let it.

7. Handoff specification

User

Print a md file (handoff or whatever you call it) so that claude code will be able to continue this job and write a working, running docker compose project. In the air-gapped environment I will be able to pull in docker images on demand. I can prepare them outside or use standard ones.

Claude

(Claude produced HANDOFF.md, delivered as a separate file. Its structure:)

1. What the system is for
2. Decisions already made (do not relitigate) — a table of the eight architectural commitments with their reasons
3. What already exists — the importer’s guarantees that downstream code can rely on
4. Repository layout to create
5. **Contracts** — project YAML shape, the full `report_model.json` schema, and the CLI contract for every tool
6. Components to build — validator, snapshot, model builder, five renderers, Makefile
7. Counting rules — settle before writing SQL
8. Docker — images, air-gap traps table, compose services, scheduling
9. Gotchas already paid for
10. Milestones M1 to M6 with acceptance criteria
11. Do not — nine prohibitions
12. Open questions for the human

Three notes on how it’s written:

The `report_model.json` contract is the load-bearing part. Get it right and the five renderers can be built and tested independently, in any order. If it drifts and renderers start querying DuckDB themselves, you get five slightly different answers to “how many projects have DB2” and spend the next month reconciling them.

The air-gap traps section is the one to read twice. Every trap listed there fails at runtime, inside the air gap, where the feedback loop is measured in days. The `--network=none` CI job catches all of them at once, and it’s worth building during M1 rather than at the end, even though it means writing it before there’s much to test.

The milestones put Excel before PDF and PPTX deliberately. It’s the output managers actually open, and the pivot-template constraint is cheap to design for and expensive to retrofit.

8. Driving Claude Code with the spec

User

How to use the md file? Create a new dir, place md in the dir, start claude code. And then? Which prompt?

Claude

Don't start from an empty directory with only the md. Claude Code will work far better with the existing code and a git history to commit into.

Setup

```
mkdir beep-catalog && cd beep-catalog
git init

mkdir -p tools templates schema reports tests/fixtures docker
cp ~/Downloads/import_xlsx.py tools/
cp ~/Downloads/make_testdata.py tools/
cp ~/Downloads/README.md tools/README-import.md
cp ~/Downloads/HANDOFF.md .

git add -A && git commit -m "importer + handoff"
```

The commit matters. Claude Code will make large multi-file changes and you want git diff to be meaningful at every step.

CLAUDE.md is the piece that makes it stick

HANDOFF.md is just a file. Claude Code won't read it unless told, and it won't survive a /clear. CLAUDE.md is loaded into every session automatically, so put the pointer there:

```
# Project: legacy-to-cloud-native catalogue & reporting
```

```
@HANDOFF.md is the spec for this repo. Read it before any task.
The Contracts, Do not and Gotchas sections are binding.
```

Working agreement:

- Build one milestone at a time. Do not start the next one unasked.
- Every tool runs in a container. `make report` must work with only Docker installed.
- Do not modify tools/import_xlsx.py unless it has a bug.
- If the open questions section blocks you, ask. Do not invent taxonomy labels.

The @HANDOFF.md syntax imports the file, so its content is in context every session without pasting it.

First session

Start in plan mode so it doesn't immediately write 2000 lines:

Read HANDOFF.md. Don't write any code yet. Tell me: what's ambiguous or underspecified, what you'd need to decide that isn't written down, and anything in the spec you think is wrong. Then propose a concrete plan for M1 only.

Worth the extra turn. It surfaces where the spec is vague, and catches the case where the model contract has been misread before that's baked into five renderers.

Then one milestone per session

Fresh context (/clear between), and a prompt shaped like this:

Implement M1 from HANDOFF.md: validate.py plus its container, Makefile target, and fixtures. Acceptance: make check exits 0 on valid fixtures and 1 on broken ones, with file:line:fix error messages. Include a fixture with a dependency cycle and one with an unknown enum code. Show me make check actually running before you say it's done.

That last sentence does real work. Without it you get code that looks right and has never been executed.

Between milestones: `git diff --stat, make check, commit`.

Two things to set up before M2

Test data. `make_testdata.py` generates a synthetic legacy sheet. Run the importer on it to produce a real projects/ tree, so Claude Code has actual data to build against rather than fixtures it invented.

Docker permissions. Add to `.claude/settings.json` so it isn't asking to approve every docker compose run:

```
{
  "permissions": {
    "allow": ["Bash(docker:*)", "Bash(docker compose:*)", "Bash(make:*)"]
  }
}
```

The prompt that matters most

At M6, be explicit, because this is the one it will otherwise fake:

Implement M6. The --network=none pipeline run is the acceptance test — actually run it and show me the output. If any of the air-gap traps fire, fix them in the Dockerfile, not by relaxing the network restriction.

Build that offline check during M1 rather than waiting for M6. It's a one-line addition to the Makefile and it stops every air-gap trap at the moment it's introduced instead of six milestones later.

9. Naming the document

User

What would be a proper name for HANDOFF.md if it were written by someone else, e.g. by me?

Claude

HANDOFF signals a one-time transfer, so a human reading the repo in six months will assume it's stale and skip it. That's the main reason to rename it: the document is meant to stay authoritative while the work is ongoing.

Given what's in it — context, binding decisions, interface contracts, build order, acceptance criteria — the closest fit is a specification.

SPEC.md at the repo root. Unambiguous, it's what an implementer looks for, and it reads as current rather than historical.

Alternatives depending on emphasis:

- **DESIGN.md** — if the “why” sections will grow faster than the contracts. Slightly softer; people treat design docs as discussable and specs as binding, and the contracts and prohibitions should be binding.

- ARCHITECTURE.md — well-established convention, but usually describes a system that exists, not one to be built. Right name later, wrong name now.
- docs/spec.md — if there will be several documents. Worth doing from the start if taxonomy.md and operations.md are already coming.
- REQUIREMENTS.md — too narrow. Half the document is implementation direction, which isn't a requirement.

In a German context there's also the framing: this is much closer to a **Pflichtenheft** than a Lastenheft, since it's the answer to how it gets built rather than a statement of what's wanted. If the document will be read by people on the project side, that word lands better than "handoff" and sets the right expectation about who owns it.

One structural note: the "decisions already made" section will accumulate entries and start crowding the contracts. When it hits a dozen, move it to docs/adr/ as one file per decision and leave a link. Contracts get edited; decisions get superseded but not deleted. Different lifecycles.

Whatever you pick, update the @HANDOFF.md import in CLAUDE.md, or none of it gets read.

Summary of decisions

Area	Decision
System of record	Git, one YAML file per project, per-team directories with CODEOWNERS
Editing	Hand-edited YAML plus schema-aware editor integration. No UI in v1
Analytical layer	DuckDB over Parquet snapshots, rebuilt every run, never hand-maintained
Report generation	One report_model.json feeds all five renderers; renderers never compute
Charts	Rendered once as PNG, embedded in PDF, PPTX and XLSX
Excel	Multi-grain sheets as real Excel tables, code plus label columns, pre-built pivots from a template, plain counts sheet
PDF	Typst; WeasyPrint as fallback
PPTX	python-pptx filling a corporate .potx
Validation	Five layers: parse, structure, referential integrity, plausibility, completeness
Validation policy	Incomplete never fails, invalid always fails
Vocabulary	No booleans. full / partial / none / unknown
Enums	In taxonomy.yaml with de/en labels, sort order and colour; not in the schema
Counting	Rules defined once in definitions.yaml, referenced by every table, printed in the output
Confidence	Raw coverage and verified coverage reported as two separate numbers
Orchestration	Scheduled GitLab CI pipeline; compose profile as fallback. Not Airflow
Air gap	Single bundle built outside; --network=none CI job as the standing regression test